HUNDRED DAYS OF CODE
DAY 30

QUESTION: Write a C program to perform the following menu-driven operations on a given array:

1. Sort the elements using Bubble Sort algorithm and perform a Binary search.
2. Perform Insertion and Deletion in a given valid position.
3. Rotate the array elements to the right by k positions.
4. Exit

SOLUTION:

1. Aim

To write a menu-driven C program using a switch statement to perform various operations on an array, including **Bubble Sort followed by Binary Search, insertion and deletion at a valid position, and right rotation of the array by k positions**.

2. Algorithm

Main Algorithm

1. Start the program.
2. Input the size of the array.
3. Input the array elements.
4. Display the following menu:
    - Bubble Sort and Binary Search
    - Insertion and Deletion
    - Right Rotation by k positions
    - Exit
5. Accept the user's choice.
6. Use a switch statement to perform the selected operation.
7. Repeat the menu until the user selects **Exit**.
8. Stop.

Operation 1: Bubble Sort and Binary Search

HUNDRED DAYS OF CODE
DAY 30

1. Traverse the array using nested loops.
2. Compare adjacent elements.
3. If the left element is greater than the right element, swap them.
4. Repeat until the entire array is sorted in ascending order.
5. Display the sorted array.
6. Input the element to be searched.
7. Set low = 0 and high = n - 1.
8. Calculate the middle position.
9. If the middle element is equal to the search element, display its position.
10. If the search element is smaller, search the left half.
11. Otherwise, search the right half.
12. Repeat until the element is found or low > high.
13. If not found, display an appropriate message.

Operation 2: Insertion and Deletion

Insertion:

1. Input the element to be inserted.
2. Input the position where it has to be inserted.
3. Check whether the position is valid.
4. Shift the elements from that position one place towards the right.
5. Insert the new element.
6. Increase the array size by 1.
7. Display the modified array.

Deletion:

1. Input the position of the element to be deleted.
2. Check whether the position is valid.
3. Shift all elements after that position one place towards the left.
4. Decrease the array size by 1.

HUNDRED DAYS OF CODE
DAY 30

5. Display the modified array.

Operation 3: Right Rotation by k Positions

1. Input the number of positions k.
2. Reduce k using k = k % n if k is greater than the array size.
3. Repeat the rotation k times.
4. Store the last element in a temporary variable.
5. Shift all remaining elements one position towards the right.
6. Place the stored last element at the first position.
7. Display the rotated array.

C PROGRAM:

```
#include <stdio.h>


#define MAX 100


int main()
{
   int arr[MAX];
   int n, i, j, choice;


   printf("Enter the number of elements: ");
   scanf("%d", &n);


   printf("Enter %d array elements:\n", n);
   for(i = 0; i < n; i++)
   {
      scanf("%d", &arr[i]);
```

HUNDRED DAYS OF CODE
DAY 30

```
}

do
{
   printf("\n----- ARRAY OPERATIONS MENU -----\n");
   printf("1. Bubble Sort and Binary Search\n");
   printf("2. Insertion and Deletion\n");
   printf("3. Right Rotation by k positions\n");
   printf("4. Exit\n");

   printf("Enter your choice: ");
   scanf("%d", &choice);

   switch(choice)
   {
      case 1:
      {
         int temp, key;
         int low, high, mid, found = 0;

         // Bubble Sort
         for(i = 0; i < n - 1; i++)
         {
            for(j = 0; j < n - i - 1; j++)
            {
               if(arr[j] > arr[j + 1])
               {
```

HUNDRED DAYS OF CODE
DAY 30

```
            temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
        }
    }
}

printf("Sorted array:\n");
for(i = 0; i < n; i++)
{
    printf("%d ", arr[i]);
}

printf("\nEnter element to search: ");
scanf("%d", &key);

// Binary Search
low = 0;
high = n - 1;

while(low <= high)
{
    mid = low + (high - low) / 2;

    if(arr[mid] == key)
    {
        printf("Element %d found at position %d.\n",
```

HUNDRED DAYS OF CODE
DAY 30

```
            key, mid + 1);
        found = 1;
        break;
      }
      else if(key < arr[mid])
      {
        high = mid - 1;
      }
      else
      {
        low = mid + 1;
      }
    }

    if(found == 0)
    {
      printf("Element not found in the array.\n");
    }

    break;
}

case 2:
{
    int subChoice, element, position;

    printf("\n1. Insertion\n");
```

HUNDRED DAYS OF CODE
DAY 30

```
printf("2. Deletion\n");
printf("Enter your choice: ");
scanf("%d", &subChoice);

switch(subChoice)
{
   case 1:
      if(n >= MAX)
      {
         printf("Array is full. Insertion not possible.\n");
         break;
      }

      printf("Enter element to insert: ");
      scanf("%d", &element);

      printf("Enter position for insertion (1 to %d): ",
           n + 1);
      scanf("%d", &position);

      if(position < 1 || position > n + 1)
      {
         printf("Invalid position!\n");
      }
      else
      {
         for(i = n; i >= position; i--)
```

HUNDRED DAYS OF CODE
DAY 30

```
    {
        arr[i] = arr[i - 1];
    }

    arr[position - 1] = element;
    n++;

    printf("Array after insertion:\n");

    for(i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");
  }

  break;

case 2:
  if(n == 0)
  {
    printf("Array is empty. Deletion not possible.\n");
    break;
  }

  printf("Enter position to delete (1 to %d): ", n);
  scanf("%d", &position);
```

HUNDRED DAYS OF CODE
DAY 30

```
    if(position < 1 || position > n)
    {
        printf("Invalid position!\n");
    }
    else
    {
        for(i = position - 1; i < n - 1; i++)
        {
            arr[i] = arr[i + 1];
        }

        n--;

        printf("Array after deletion:\n");

        for(i = 0; i < n; i++)
        {
            printf("%d ", arr[i]);
        }
        printf("\n");
    }

    break;

default:
    printf("Invalid choice!\n");
```

```
    }

    break;
}

case 3:
{
    int k, temp;

    if(n == 0)
    {
        printf("Array is empty.\n");
        break;
    }

    printf("Enter number of positions to rotate: ");
    scanf("%d", &k);

    k = k % n;

    // Rotate right k times
    for(i = 0; i < k; i++)
    {
        temp = arr[n - 1];

        for(j = n - 1; j > 0; j--)
        {
```

```
            arr[j] = arr[j - 1];
          }

          arr[0] = temp;
        }

        printf("Array after right rotation:\n");

        for(i = 0; i < n; i++)
        {
          printf("%d ", arr[i]);
        }

        printf("\n");

        break;
      }

      case 4:
        printf("Program exited successfully.\n");
        break;

      default:
        printf("Invalid choice! Please try again.\n");
    }

} while(choice != 4);
```

HUNDRED DAYS OF CODE
DAY 30

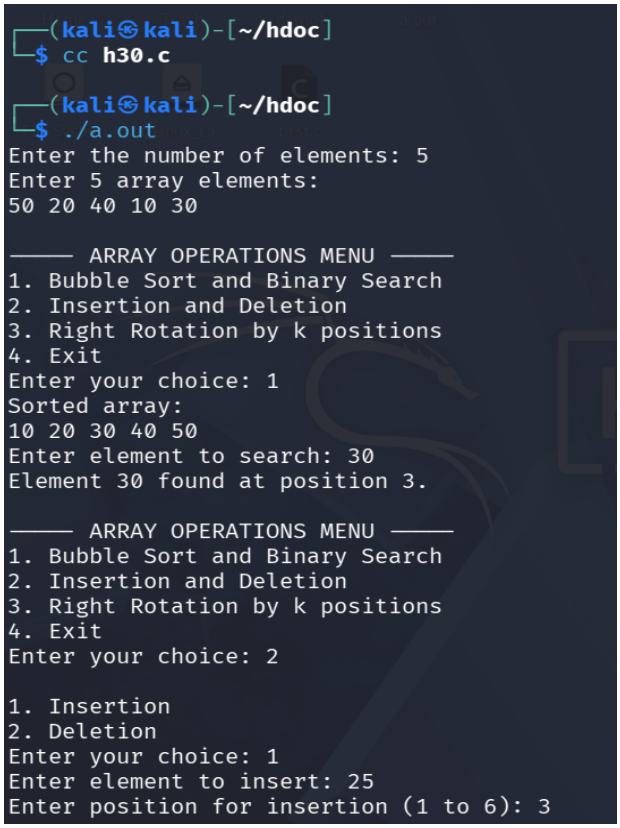
```
    return 0;
}
```

TEST CASE TABLE;

Test Case	Operation	Input	Expected Result
1	Bubble Sort	5: 50 20 40 10 30	10 20 30 40 50
2	Binary Search	Search 30 in sorted array	Found at position 3
3	Binary Search	Search 70	Element not found
4	Insertion	10 20 30 40, insert 25 at position 3	10 20 25 30 40
5	Deletion	10 20 30 40, delete position 2	10 30 40
6	Right Rotation	1 2 3 4 5, k = 2	4 5 1 2 3
7	Invalid Position	Insert at position 10 in array of size 5	Invalid position
8	Exit	Choice = 4	Program exits successfully

HUNDRED DAYS OF CODE
DAY 30

COMPILATION AND EXECUTION:



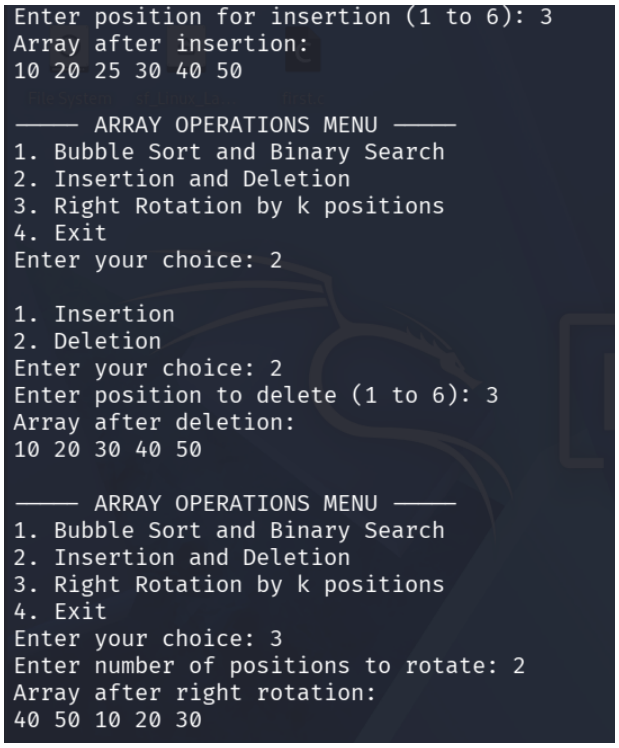
```
┌──(kali㉿kali)-[~/hdoc]
└─$ cc h30.c

┌──(kali㉿kali)-[~/hdoc]
└─$ ./a.out
Enter the number of elements: 5
Enter 5 array elements:
50 20 40 10 30

——— ARRAY OPERATIONS MENU ———
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Right Rotation by k positions
4. Exit
Enter your choice: 1
Sorted array:
10 20 30 40 50
Enter element to search: 30
Element 30 found at position 3.

——— ARRAY OPERATIONS MENU ———
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Right Rotation by k positions
4. Exit
Enter your choice: 2

1. Insertion
2. Deletion
Enter your choice: 1
Enter element to insert: 25
Enter position for insertion (1 to 6): 3
```



```
Enter position for insertion (1 to 6): 3
Array after insertion:
10 20 25 30 40 50

——— ARRAY OPERATIONS MENU ———
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Right Rotation by k positions
4. Exit
Enter your choice: 2

1. Insertion
2. Deletion
Enter your choice: 2
Enter position to delete (1 to 6): 3
Array after deletion:
10 20 30 40 50

——— ARRAY OPERATIONS MENU ———
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Right Rotation by k positions
4. Exit
Enter your choice: 3
Enter number of positions to rotate: 2
Array after right rotation:
40 50 10 20 30
```